

LENGUAJE de Máquina y Ensamblador

UNIDAD III – Segunda Parte

P
R
O
G
R
A
M
A
D
O
R

P
R
O
G
R
A
M
A
D
O
R

D
E
S
I
S
T
E
M
A

Nivel 5

NIVEL DE LENGUAJES
ORIENTADOS A PROBLEMAS

Nivel 4

NIVEL DE LENGUAJE
ENSAMBLADOR

Traducción (Compilador)

Nivel 3

NIVEL DE MAQUINA DEL
SISTEMA OPERATIVO

Traducción (Ensamblador)

Nivel 2

NIVEL DE MAQUINA DEL
SISTEMA CONVENCIONAL

*Interpretación Parcial
(Sistema Operativo)*

Nivel 1

NIVEL DE
MICROPROGRAMACIÓN

Interpretación (Microprograma)

Nivel 0

NIVEL DE LÓGICA DIGITAL

*Microprogramas ejecutados
directamente por el Hardware*

En el nivel de lógica digital los objetos que intervienen se denominan compuertas y constituyen los dispositivos digitales que resuelven las situaciones lógicas.

ASSEMBLER

Es un lenguaje de programación de nivel intermedio para los computadores, microprocesadores, microcontroladores y otros circuitos integrados programables.

Hay una correspondencia biunívoca entre las instrucciones de máquina y las sentencias del programa. Y tiene acceso a todos los recursos e instrucciones de la máquina.

TRADUCTORES

Los programas que convierten un programa escrito en un lenguaje a otro distinto se llaman traductores. El lenguaje original se llama fuente y el que se convierte, lenguaje objeto. Ambos definen niveles.

La traducción se usa cuando se dispone de un procesador para el lenguaje objeto y no se cuenta con un procesador para el programa fuente.

COMPILADOR – Interpretación

En **la Compilación**, el programa original, escrito en lenguaje fuente, no se ejecuta directamente. Se convierte en un programa equivalente, llamado programa objeto, el que se ejecuta después que se ha terminado la traducción. Hay 2 pasos:

- 1-Generación de un programa en módulo Objeto
- 2-Ejecución del nuevo programa.

En la Interpretación hay un solo paso, la ejecución del programa fuente original

NIVELES DE LENGUAJE

- **Lenguaje binario absoluto**
- **Lenguaje Assembler o simbólico de máquina**-el traductor es el programa **Ensamblador**
- **Lenguaje de Alto Nivel**. El traductor es el programa **Compilador**
- Los dos primeros son lenguajes particulares de cada máquina, mientras que los de alto nivel son independientes del hardware.

FORMATO DEL ASSEMBLER

Presenta 4 partes o campos:

- Etiqueta: Dá nombres simbólicos a las direcciones
- Operación: Abreviatura simbólica del Código de Operación.
- Operandos: Dice qué va a sumar a qué
- Comentarios: Explica el funcionamiento del programa, para que otros programadores lo usen o lo modifiquen.

EJEMPLO

Alto nivel	Assembler	Absoluto
20 z=A+B+C-D	LA D, (3 A)	00100110
		00111010
	AD A (3 B)	01010001
		01011011
	AD A (3 C)	11001000
		00111100

Seudo Instrucciones

- Las pseudoinstrucciones son directivas al Ensamblador, que no se traducirá a código de máquina.
- Ejemplo la palabra FIN.
- El ensamblador se encargará de asignar a cada etiqueta una dirección en la zona de datos.

PROCESO DE ENSAMBLAJE

- Referencia adelantada: por ej.” Salto a L”
- Se soluciona de 2 maneras:
- 1- Leer 2 veces el programa fuente (traductor de 2 pasadas)
- 2- Leer una sola vez el programa Fuente (traductor de 1 pasada)

ENSAMBLAJE DE DOS PASADAS

- En la primera lectura detecta todas las etiquetas definidas, en memoria forma una tabla con ellas. Crea la “tabla de Símbolos”, donde se guardan los valores de todos los símbolos (asociar una dirección en memoria).
- En la segunda lectura (pasada), se conocen los valores de todos los símbolos, se puede leer cada sentencia, ensamblarla y obtener una salida.

ENSAMBLAJE DE UNA PASADA

- Se introduce la sentencia con la referencia adelantada en una tabla, con la indicación que no ha sido ensamblada. Al final todos los símbolos habrán sido definidos y pueden ensamblarse.
- El problema de una pasada es que si hay muchas sentencias con ref. adelantadas, la tabla que las contiene puede ser muy grande, y no caber en memoria.

En la primera lectura

- Por lo general la mayoría de los assemblers construyen por lo menos 2 tablas:
 - La tabla de símbolos
 - La tabla de códigos de operación
- Y en caso de usar literales
 - La tabla de literales.

TABLA DE SÍMBOLOS

- Tiene un elemento para cada símbolo. Los símbolos se definen, usándolos como etiquetas.
- Cada elemento de la tabla contiene:
 - -El símbolo (o un apuntador a él)
 - - Su valor numérico.
- Y algunas veces otra información como por ej.
- Longitud del campo, bits de ubicación, etc.

TABLA DE SÍMBOLOS

- Símbolo ----- Valor-----Otra información
- PEDRO ----- 100 ----- Long. del campo
- JUAN ----- 111 ----- Bits de reubicación
- JOSE ----- 125 ----- Símbolo accesible o no

ORGANIZAR LA TABLA DE SIMBOLOS

- Vector de Pares: el 1er elemento es el símbolo y el 2do es el valor. Dado un símbolo a buscar, la rutina de la tabla busca en forma literal hasta que lo encuentra. Es fácil de programar pero es lento.
- Búsqueda binaria: hay que ordenar los símbolos. El algoritmo funciona comparando el elemento central con el símbolo.

Algoritmo de búsqueda binaria

- Si se localiza en orden alfabético por delante de aquél, hay que proseguir la búsqueda en la 1ra mitad de la tabla, y si está después, en la 2da mitad. Si es igual la búsqueda termina.
- Este método es más rápido que la búsqueda lineal, pero requiere primero ordenar la tabla

Codificación por Transformación

- Requiere de un función de transformación que haga corresponder a cada símbolo un nro entre 0 y $k-1$. Ejemplo
- SIMBOLO VALOR Código de transformación

JUAN	14025	0
PEDRO	45012	3
JOSE	34004	2
MARIA	45019	0

MACROINSTRUCCIONES

Por lo general. en un programa hay instrucciones que se repiten en el programa fuente. El lenguaje Assembler las simplifica, acortando la escritura. **Una alternativa** es convertir la secuencia en un procedimiento y llamarlo cada vez que se necesite. El inconveniente es que se requiere una instrucción de llamada y otra de retorno cada vez que se ejecute la secuencia (disminuye la velocidad)

Macroinstrucciones

Representan otra alternativa son las Macroinstrucciones:

- El programador define un módulo de programa y le asigna un nombre, que escribirá en lugar de la secuencia cada vez que aparezca esa sucesión. El programa traductor se llama Macro-Assembler, el cuál antes de traducir realiza la expansión de las macros, o sea cada vez que se detecta una macro la sustituye por la secuencia o módulo.

Macroprocesamiento

- El proceso por el cual en un texto se sustituye un grupo de símbolos por otro se llama “**macroprocesamiento**”.
- Cada vez que el programador necesite definir una nueva macro, deberá realizar al comienzo del programa una **macrodefinición**, con el nombre de la misma y detalle de las instrucciones que comprende el módulo.

EJEMPLO

Supongamos que se repite una suma a lo largo del programa. Se lo define con la macro SUMBYTES:

```
DEFM    SUMBYTES  
        LDA,BYTE L1  
        ADA,BYTE L2  
        STA, BYTE L  
        BCO, BETA
```

MACROS-SUBROUTINAS

- Es posible que en muchas macro-assembler, que un macro llame a otra y de esta a otra, resultando un enlazado de macros.
- En los lenguajes de alto nivel se utilizan las subrutinas. Estas ahorran memoria, ya que son escritas una sola vez, como subprogramas independientes en código de máquina, y son llamadas durante la ejecución (ya traducido).

MACRO-SUBROUTINAS

- A diferencia de las subrutinas, las macros son llamadas antes que el programa se traduzca, y se incorporan tantas veces como se escribieron en el programa fuente.
- En síntesis, tanto las macros como las subrutinas permiten que el programador no escriba secuencias de instrucciones repetidas

PARTES DE UNA MACRO

- **Un encabezado:** proporciona el nombre
- **El texto:** forma el cuerpo de la macro
- **Una seudoinstrucción:** marca el fin de la definición.
- El programador forma una tabla de macros para su uso a medida que las encuentra.
- El ensamblador las reemplaza por su cuerpo, siempre que aparezca como cod.de operación.

Macros-Procedimientos

- Llamadas a Macro: indican al ensamblador que reemplace el nombre de la macro por su cuerpo. (se expanden en la 1ra pasada del proceso de ensamblaje).
- Llamadas a procedimientos: es una instrucción de máquina que será insertada en el programa objeto, y más tarde será ejecutada para llamar al procedimiento.

Implementación

El ensamblador debe:

- Guardar las definiciones
- Expandir las definiciones

Las definiciones son guardadas en una tabla de macros y junto con cada nombre un apuntador a otra tabla donde se guarda el cuerpo.

Programas: Cargador o Editor de Enlaces

Antes de ejecutarse el programa, todos los procedimientos traducidos deben recuperarse y ligarse o enlazarse correctamente. Si no se cuenta con memoria virtual, debe cargarse explícitamente en memoria. Los programas encargados de esta tarea se denominan: cargador, montador o enlazador, entre otros.

Traducción completa

La traducción del programa fuente se la realiza en dos (2) pasos:

- 1 - COMPILACIÓN O ENSAMBLAJE, de los procedimientos fuente.
- 2 - ENCADENAMIENTO O MONTAJE, de los módulos objetos.

REUBICACIÓN

- Construye una tabla de todos los módulos objeto y sus longitudes.
- Asigna una dirección de carga a cada módulo objeto.
- A las instrucciones que contengan una dirección le suma una constante de reubicación, igual a la dirección de comienzo del módulo en que están contenidas.
- Busca todas las instrucciones que hacen referencias a otros procedimientos e inserta en ella la dirección.

EJEMPLO

Módulo	Longitud	Dirección de comienzo
A	400	100
B	600	500
C	500	1100
D	300	1600

Estructura de un módulo Objeto

Identificación

Tabla de puntos de Entrada

Tabla de Referencias Externas

Instrucciones de máquina y constantes

Lista de reubicación

Fin del Módulo

Dudas y consultas...